# Banking Loan Risk Analysis Application

System Design Document, Architecture & Technical Trade-offs

---

## 1. Problem Statement & Business Objective

Financial institutions manage tens of thousands of credit card lines and retail loan accounts. Credit analysts face severe operational inefficiencies when monitoring exposure using spreadsheets. This project implements an enterprise full-stack banking application providing automated batch ETL, pure MySQL relational storage, transparent rule-based risk band classifications (Low, Medium, High), underwriter manual review workflows, role-based REST APIs, and an interactive Streamlit portal.

## 2. High-Level Architecture Design & Microservice Tiers

The system is designed with four decoupled tiers:

- **Data Ingestion & Batch Lifecycle:** New data batches (.xls, .xlsx, .csv) are placed into data/raw/. The pipeline extracts, validates, and transforms data, commits records to MySQL, verifies MD5 checksum reconciliation, and automatically moves processed files into data/processed/ with timestamp prefixes.
- **Relational Data Mart (MySQL Only):** 3NF normalized schema in loan_risk_db comprising customers, credit_profiles, bills, payments, data_quality_logs, etl_reconciliation_logs, and a materialized segment_kpi_summaries table indexed for sub-millisecond query performance.
- **Business Rules & Risk Engine:** Multi-factor risk scoring (0-100 scale), Low/Medium/High risk band mapping, and automated underwriter review policy exception triggers.
- **API & User Presentation Layer:** FastAPI ASGI service with OAuth2/JWT role-based access control (Administrator vs Analyst), serving a multi-tab Streamlit dashboard with dedicated login and live What-If loan underwriting simulators.

## 3. Technology Selection Justification & Trade-Offs

- **Pure MySQL:** Chosen over embedded databases for production ACID guarantees, strict foreign key constraints, connection pooling, and multi-user transactional concurrency.
- **FastAPI vs Flask:** FastAPI provides native asynchronous ASGI performance, automatic OpenAPI/Swagger documentation, and Pydantic V2 schema validation.
- **Pandas & SQLAlchemy ORM:** Efficient vectorized data cleaning paired with safe type-mapped ORM transactions and connection pooling.
- **Streamlit & Plotly:** Reactive UI allowing credit analysts and directors to evaluate scenarios and manage review queues with zero frontend overhead.

## 4. Business Rules & Risk Band Thresholds

- **High Risk (Score ≥ 65):** Max repayment delay ≥ 2 months, OR utilization ≥ 85% with repayment ratio < 20%, OR historical default.
- **Medium Risk (Score 35–64):** Max repayment delay = 1 month, OR utilization 60%–85%, OR repayment ratio 20%–50%.
- **Low Risk (Score < 35):** Max repayment delay ≤ 0, utilization < 60%, repayment ratio ≥ 50%.
- **Manual Review Queue Triggers:** Limit > NT00,000 with payment delay; utilization > 90% with repayment < 15%; chronic delinquency in 2+ cycles; statement > NT0,000 with zero payment.

## 5. Security, RBAC & Batch Lifecycle Decisions

- **JWT Security:** State-guarded sign-in view requiring valid credentials. Issued tokens enforce role boundaries (e.g. only Administrators can execute ETL pipelines).
- **Automated Data Archiving:** Strict separation of data/raw/ and data/processed/ ensures idempotency, preventing accidental double-processing of data batches.